

Relatório de Desenvolvimento

VettiFlow 1.0 — Gestão de Fluxo de Produção

Emitido em 23 de julho de 2026 | Branch **master** | Último commit **caff6dd**
(03/07/2026)

Resumo executivo

O produto está **funcionalmente maduro**: o fluxo completo de chão de fábrica está modelado, testado e com interface polida em quatro plataformas. Porém, hoje ele é um **protótipo de máquina única** — cada estação guarda seus próprios dados localmente e não enxerga as demais. O único passo que separa o estado atual de um piloto real é o **backend compartilhado**.

1. Situação geral

Indicador	Estado
Código Dart	55 arquivos — aproximadamente 25.988 linhas
Análise estática (<i>flutter analyze</i>)	Sem problemas encontrados
Testes automatizados (<i>flutter test</i>)	22 de 22 aprovados
Plataformas suportadas	Windows, Web, Android e macOS (base de código única)
Arquitetura	flutter_bloc (Cubit) + Provider + padrão Repository
Commits no histórico	11 commits desde 24/06/2026

2. O que já foi entregue

2.1 Fluxo de produção completo — 7 etapas

O modelo `ProductionStage` cobre o ciclo inteiro da OP: **Almoxarifado** → **Gravação** → **Soldagem** → **Teste** → **Fechamento** → **Expedição** → **Armazenada/Finalizada**. Cada etapa possui tela dedicada, rota registrada e regras próprias de conclusão.

Tela / Módulo	Arquivo	Linhas
Expedição	ui/expedition/expedition_page.dart	3.019
Teste	ui/testing/testing_page.dart	2.195
Soldagem	ui/soldering/soldering_page.dart	1.643
Almoxarifado	ui/warehouse/warehouse_page.dart	1.453
Relatórios	ui/dashboard/views/reports_view.dart	1.444

Tela / Módulo	Arquivo	Linhas
Painel de detalhe da OP	ui/dashboard/widgets/op_detail_panel.dart	1.257
Fechamento	ui/closing/closing_page.dart	1.216
VettiFlow TV	ui/tv/vetti_flow_tv_page.dart	1.122
Dashboard	ui/dashboard/dashboard_page.dart	1.001
Suporte	ui/support/support_page.dart	890
Gravação (firmware)	ui/firmware/firmware_page.dart (+635 de diálogos)	739

2.2 Motor de estado da produção

O `ProductionFlowStore` (885 linhas) é o coração da aplicação. Ele concentra todas as transições de estado das ordens:

- **Ciclo da etapa:** `startStage`, `pauseStage`, `completeStage`, `regressStage`, `resetStage`, `cancelOrder`.
- **Encerramentos específicos:** `completeTesting` (com registro de defeitos), `completeClosing` (quantidade fechada), `completeExpedition` e `dispatchStored` (baixa parcial de estoque).
- **Multi-operador por etapa:** vários colaboradores podem atuar na mesma OP; a etapa só avança quando todos os operadores ativos concluem — comportamento coberto por teste.
- **Auditoria de pausas:** motivo, PIN do operador, horário e quantidade são gravados; o motivo “outro” exige justificativa escrita — também coberto por teste.

2.3 Login e gestão de equipe por setor

Autenticação real contra o `OperatorAssignmentStore`, com redirecionamento automático do usuário para a tela do seu setor. O cadastro contempla cerca de 30 operadores com nome, usuário, PIN, papel, área de atuação (Produção, SMD, Almoxarifado, Suporte, Sistema) e permissão de gestão de atribuições.

2.4 Painel administrativo e relatórios

- **Dashboard:** visões Kanban, Tabela, Cards, Ordens, Armazenadas e Atribuição de operadores, com KPIs, barra de filtros e painel lateral de detalhe da OP.
- **Relatórios:** tempo por etapa, tipos de defeito, eventos de pausa, produtividade por operador e quebra por produto.
- **VettiFlow TV:** painel rotativo para exibição em chão de fábrica, com testes de rotação de painéis e de viewport compacto.

2.5 Base técnica

- **Responsividade:** `AppBreakpoints` com layouts desktop e mobile separados em todas as telas principais, incluindo navegação inferior no mobile.
- **Persistência (Web):** snapshot em JSON no `localStorage`, com sincronização entre abas via evento `storage`.

- **Qualidade:** análise estática limpa e 22 testes automatizados cobrindo navegação, renderização por viewport e regras do fluxo de produção.

3. Gargalos que travam o piloto

Os pontos abaixo estão ordenados por impacto. Os três primeiros são bloqueadores para uso em produção real.

#	Gargalo	Impacto
1	Persistência existe apenas na Web	A implementação para desktop e mobile é um <i>stub</i> vazio: retorna nulo e descarta escritas. No Windows, macOS e Android todo o histórico de produção se perde ao fechar o aplicativo .
2	Não há estado compartilhado entre máquinas	Cada estação possui seu próprio armazenamento local. O Almoxarifado não enxerga a OP que a Soldagem iniciou. Este é o bloqueio central : o app é multi-tela, mas ainda não é multi-usuário de fato.
3	Senhas em texto plano no código-fonte	Usuário, senha e PIN de cada operador estão literais e versionados no Git (há casos em que a senha é igual ao próprio PIN). Qualquer mudança de equipe exige recompilar e redistribuir o aplicativo.
4	Integração com o Protheus não iniciada	A abstração <code>OpRepository</code> está pronta e enxuta (29 linhas), mas só existe a implementação <i>mock</i> . O <code>ProtheusOpRepository</code> ainda não foi escrito.
5	Telas grandes demais	Expedição (3.019 linhas) e Teste (2.195 linhas) concentram interface, regra de negócio e estado no mesmo arquivo. Ainda navegável, porém cada alteração fica progressivamente mais cara.

4. Próximos passos recomendados

Fase 1 — Tornar o piloto viável (prioridade máxima)

Ação	Descrição	Prioridade
Backend compartilhado	Sem isso, nada mais importa. Recomendação: API REST leve (FastAPI ou Node) com PostgreSQL, expondo exatamente o formato JSON que a serialização atual já produz. Alternativa mais rápida: Supabase, que já entrega tempo real e autenticação prontos.	Crítica
Migrar o store para a API	Substituir o armazenamento local por um cliente de API, mantendo o cache local para operação offline. A superfície de mudança é pequena: os métodos públicos do store permanecem os mesmos.	Crítica

Ação	Descrição	Prioridade
Persistência em desktop e mobile	Implementar o <i>stub</i> com <i>shared_preferences</i> ou <i>path_provider</i> . Cerca de meio dia de trabalho e resolve imediatamente a perda de dados.	Alta

Fase 2 — Segurança e operação

Ação	Descrição	Prioridade
Remover credenciais do código	Usuários e <i>hashes</i> de senha no banco, com autenticação via endpoint. Enquanto isso não ocorrer, o aplicativo deve ser tratado como apto somente a rede interna confiável.	Alta
Integração com o Protheus	Implementar o <i>ProtheusOpRepository</i> sobre a abstração já existente. Começar pela leitura de OPs — só a importação já entrega valor — e depois evoluir para o retorno de apontamentos.	Média

Fase 3 — Sustentação

Ação	Descrição	Prioridade
Modularizar as telas grandes	Extrair widgets e lógica de Expedição e Teste para subpastas <i>widgets/</i> , seguindo o padrão já adotado no módulo de Gravação.	Média
Ampliar a cobertura de testes	Os 22 testes atuais cobrem navegação e o caminho feliz. Faltam cenários de erro, concorrência entre operadores e serialização/desserialização do store.	Média
Exportação de relatórios	Saída em PDF e Excel. A tela de relatórios já calcula todos os indicadores — resta apenas a camada de exportação.	Baixa

Conclusão

A engenharia de produto está em bom estado: código limpo, testes verdes, arquitetura com as abstrações certas no lugar. O esforço restante não é de funcionalidade, e sim de **infraestrutura**. A decisão imediata é apenas de sequência: começar pela persistência local — rápida e que tapa o risco de perda de dados — ou partir direto para o desenho do backend compartilhado.